TASKMANAGER FINAL PROJECT DOCUMENT

Agile Software Development & DevOps — Semester Project Repository: https://github.com/AndyDebrah/taskmanager **Technology:** Java 17, Spring Boot 3, JWT (jjwt), H2, Maven, GitHub Actions, JaCoCo, Actuator, Swagger **Methodology:** Agile Scrum — Sprint 0 (Planning), Sprint 1 & Sprint 2 (Execution)

1. Executive Summary

The Taskmanager project is a functional Task Management prototype developed as the central deliverable of an Agile Software Development and DevOps semester course. The system allows registered users to create, retrieve, update, and delete personal tasks through a secured REST API, supported by a lightweight browser-based interface. Authentication is enforced across all task operations using JSON Web Tokens (JWT), issued on successful login and validated on every subsequent request through a stateless Spring Security filter chain.

Development was organised into three phases: Sprint 0 (planning, backlog refinement, architecture design), Sprint 1 (backend API, security, testing, CI scaffolding), and Sprint 2 (UI, observability, documentation, coverage reporting). Technologies used include Java 17, Spring Boot 3, jjwt, H2, Maven, GitHub Actions, JaCoCo, Spring Boot Actuator, Springdoc OpenAPI, and a vanilla HTML/JavaScript frontend. The project demonstrates how Agile ceremonies and DevOps automation combine to produce a working, observable, and maintainable system within a constrained academic timeline.

---

2. Product Vision & Initial Planning (Sprint 0)

2.1 Product Vision

"To provide individual users with a secure, lightweight, and intuitive task management tool that enables them to organise their work through a reliable REST API backed by JWT authentication, accessible from any browser."

Scope was deliberately constrained so that every committed feature could be delivered with full automated test coverage and a functioning CI pipeline — embodying the Agile principle of working software over comprehensive documentation.

2.2 Scope & Epics

Epic	Description
User Authentication	Registration, login, JWT issuance, and stateless security enforcement across all protected endpoints.

Task Management	Full CRUD (create, list, update status, delete) scoped to the authenticated user.
UI Frontend	Static HTML/JS login page and task dashboard, no external framework.
DevOps / CI/CD	GitHub Actions pipeline, Dockerfile, Maven caching, JaCoCo coverage in CI.
Monitoring & Documentation	Actuator health endpoint, structured logging, Springdoc OpenAPI/Swagger.

2

.3 Full Product Backlog

ID	User Story	Acceptance Criteria	Priority	Est.	Sprint
US-01	As a user I want to register with a username and password so I can have a personal account.	POST /register returns 200 with username. Duplicate returns 409.	High	3	SP1
US-02	As a user I want to log in and receive a JWT token so I can call protected endpoints.	POST /login returns {token}. Invalid credentials return 401.	High	3	SP1
US-03	As a user I want my password stored securely so my account cannot be compromised.	Passwords are BCrypt-hashed before storage. Plain text never persisted.	High	2	SP1
US-04	As a user I want to create a task with a title and description	POST /api/tasks returns 201 with task JSON including id, status, ownerId.	High	2	SP1

	so I can track my work.				
US-05	As a user I want to list all my tasks so I can see my current workload.	GET /api/tasks returns only the authenticated user's tasks.	High	2	SP1
US-06	As a user I want to update a task's status so I can track progress.	PUT /api/tasks/{id}?status=COMPLETED updates status and returns 200.	High	2	SP1
US-07	As a developer I want a CI pipeline so tests run automatically on every push.	GitHub Actions runs mvn package and mvn test on every push to main.	High	3	SP1
US-08	As an operator I want a Dockerfile so the application can be containerised.	docker build produces a runnable image; docker run starts app on port 8080.	Med	2	SP1
US-09	As a user I want a login UI so I can authenticate without console token injection.	login.html stores the returned JWT in localStorage automatically.	High	5	SP2
US-10	As a user I want a task dashboard so I can manage tasks from the browser.	dashboard.html supports create, update, delete. Redirects to login if token absent.	High	5	SP2
US-11	As a user I want to delete a task so I can remove irrelevant items.	DELETE /api/tasks/{id} returns 204. Cross-user deletion returns 403.	High	3	SP2
US-12	As an operator I want a health endpoint so I can monitor service availability.	GET /actuator/health returns {status:UP} without authentication.	Med	1	SP2

US-13	As a developer I want OpenAPI/Swagger docs so the API is self-describing.	Swagger UI at /swagger-ui.html lists all endpoints with schemas.	Med	2	SP2
US-14	As a developer I want coverage reporting in CI so I can track test quality.	JaCoCo generates an HTML report in CI uploaded as a build artifact.	Med	2	SP2

2.4 Definition of Done

An item was not considered complete unless all of the following criteria were met:

1. All acceptance criteria are fully implemented and verified.
2. Code is reviewed, clean, and merged to the main branch.
3. Unit and/or integration tests cover the new functionality and all tests pass.
4. No pre-existing tests are broken (zero regressions).
5. The CI pipeline passes the build and test stages.
6. API changes are documented via inline comments or OpenAPI annotations.
7. The feature is demonstrable end-to-end via curl, the UI, or an automated test.
8. Relevant retrospective action items from the prior sprint have been addressed.

2.5 Architecture Overview

The system follows a layered, REST-oriented architecture built on Spring Boot.

Presentation Layer: Two static HTML files served by the embedded Tomcat server. login.html submits credentials via AJAX and stores the returned JWT in localStorage. dashboard.html reads the token and issues all subsequent API calls with an Authorization: Bearer header. All interactivity is implemented in vanilla JavaScript.

API / Controller Layer: AuthController exposes /api/auth/register and /api/auth/login, handling BCrypt password hashing and JWT issuance. TaskController exposes /api/tasks for task CRUD. Both controllers delegate all business logic to the service layer.

Service Layer: TaskService encapsulates business rules — associating tasks with their owning user, enforcing owner-only update and delete, and mapping between entities and response objects.

Repository / Persistence Layer: Spring Data JPA repositories over an H2 in-memory database. Replacing the H2 datasource configuration with PostgreSQL or MySQL requires no code changes.

Security Layer: A stateless JwtAuthenticationFilter intercepts all /api/ requests, validates the Bearer token signature and expiry using HS256, and populates the Spring Security context. The /api/auth/ endpoints are explicitly permitted without authentication. The JWT secret is configured in application.properties for development, with a documented migration path to environment variables for production.

Monitoring & Observability: Spring Boot Actuator exposes /actuator/health. Structured SLF4J logging covers login events, task operations, authorisation failures, and unhandled exceptions. Springdoc auto-generates Swagger UI at /swagger-ui.html.

CI/CD Pipeline: A GitHub Actions workflow in .github/workflows/ci.yml triggers on every push and pull request to main. It checks out the repository, installs JDK 17, caches Maven dependencies, runs mvn package and mvn test, and uploads the JaCoCo HTML coverage report as a build artifact.

3. Sprint 1 — Implementation & CI/CD Foundation

3.1 Sprint 1 Goal

Deliver a working Task Manager backend with JWT authentication, task create/list/update endpoints, comprehensive unit and integration tests, a GitHub Actions CI workflow, and Docker packaging — a complete, testable, containerisable REST API prototype.

Sprint 1 committed 17 story points across US-01 through US-08. The frontend UI and E2E tests were deliberately deferred to Sprint 2 to ensure the backend was thoroughly validated first.

3.2 Deliverables Completed

User Registration & Login: POST /api/auth/register validates uniqueness, hashes the password with BCrypt, and returns HTTP 200 with the username. POST /api/auth/login verifies credentials against the BCrypt hash and returns HTTP 200 with a signed JWT on success, or HTTP 401 on failure.

JWT Security Filter: JwtAuthenticationFilter extends OncePerRequestFilter. For every /api/ request it extracts the Bearer token, validates signature and expiry, loads

UserDetails, and populates the SecurityContextHolder. Requests without a valid token receive HTTP 401 before reaching a controller.

Task CRUD Endpoints: POST /api/tasks creates a task owned by the authenticated user (HTTP 201, PENDING status). GET /api/tasks returns only the calling user's tasks (data isolation). PUT /api/tasks/{id}?status=COMPLETED validates ownership and returns HTTP 200, or HTTP 403 if the task belongs to another user.

Exception Handling: A global @ControllerAdvice maps ResourceNotFoundException to 404, UnauthorisedException to 403, and DuplicateUsernameException to 409, ensuring consistent error responses with no stack trace leakage.

Tests: AuthControllerIntegrationTest covers successful registration, duplicate rejection, successful login, and invalid credential rejection through MockMvc. TaskControllerIntegrationTest covers task creation, listing, status update, data isolation, and unauthenticated access rejection. TaskServiceTest provides unit tests with Mockito-mocked repositories for all service paths. A test-data collision issue (shared usernames across test methods) was resolved with UUID-suffixed usernames per test.

CI Pipeline: .github/workflows/ci.yml triggers on push and pull_request to main. Steps: checkout → setup JDK 17 (Temurin) → cache ~/.m2 → mvn package → mvn test → upload JaCoCo report as artifact.

Dockerfile: Two-stage build. The Maven stage compiles and packages the JAR. The slim JRE 17 runtime stage copies the JAR, exposes port 8080, and sets the entrypoint.

3.3 Sprint 1 Scope Status

Deliverable	**Status**
POST /api/auth/register with BCrypt hashing	Done
POST /api/auth/login with JWT issuance	Done
JWT authentication filter — stateless Spring Security	Done
POST /api/tasks	Done
GET /api/tasks — user-scoped	Done
PUT /api/tasks/{id} — update status	Done
Global exception handler	Done
AuthController and TaskController integration tests	Done

TaskService unit tests	Done
GitHub Actions CI workflow	Done
Dockerfile	Done
Frontend login UI with localStorage token persistence	Not Done — deferred SP2
DELETE /api/tasks/{id}	Not Done — deferred SP2
JaCoCo coverage reporting in CI	Not Done — deferred SP2

3.4 CI/CD Evidence

Repository: https://github.com/AndyDebrah/taskmanager Actions page: https://github.com/AndyDebrah/taskmanager/actions

Most recent recorded run:

- Run ID: 22070252613
- URL: https://github.com/AndyDebrah/taskmanager/actions/runs/22070252613
- Date/Time (UTC): 2026-02-16T16:21:02Z
- Status: Failure

The failure is caused by a JaCoCo 0.8.10 bytecode instrumentation incompatibility with JDK versions newer than 21 on the GitHub-hosted runner. The fix — pinning the workflow to JDK 17 or adding -Djacoco.skip=true — is tracked as an immediate action item. All tests pass locally under JDK 17 with mvn -Djacoco.skip=true test.

3.5 Sprint 1 Review Summary

All Sprint 1 committed stories met their acceptance criteria. The review demonstrated: registration and login via curl with a live JWT in the response body; task create, list, and update via curl with Bearer token authentication; the full test suite executing locally; and the Docker image running on port 8080.

Deferred to Sprint 2: frontend login UI, DELETE endpoint, Actuator, Swagger, JaCoCo in CI.

Risks identified:

Risk	Mitigation
JWT secret hardcoded in application.properties	Move to environment variable before production.
JaCoCo/JDK incompatibility causing CI failure	Pin runner to JDK 17. Tracked as immediate action item.
No production deployment pipeline	Define IaC and CD in Sprint 2.
No E2E test coverage	Add targeted E2E tests in Sprint 2.

3.6 Sprint 1 Retrospective

What Went Well:

- All backend endpoints implemented and verified by automated tests before any UI work began.
- JWT security confirmed working via integration tests, not just manual testing.
- Tests caught the test-data collision early; resolved within the sprint.
- Dockerfile and CI workflow scaffolded in the same sprint as the application.
- Fully demonstrable API via curl by sprint end.

What Didn't Go Well:

- Test-data collisions caused unexpected 409 failures until UUID-suffixed usernames were adopted.
- CI was not re-run after the final documentation commit, leaving a failure in the run history.
- JWT secret hardcoded in application.properties.
- Dashboard required manual browser console token injection — poor UX.

Sprint 1 Action Items:

Action Item	Owner	Due
Use UUID-suffixed usernames in tests to prevent data collisions	dev	Sprint 2 start
Implement login.html with localStorage JWT persistence	dev	Sprint 2
Move JWT secret to environment variable	dev / ops	Sprint 2
Add E2E tests for auth and task flows	dev	Sprint 2
Pin GitHub Actions runner to JDK 17 to resolve CI failure	dev / ops	Immediate

4. Sprint 2 — Execution & Improvement

4.1 Sprint 2 Goal

Extend the prototype with a functional browser login UI, delete-task API with owner enforcement, Spring Boot Actuator health monitoring, Springdoc OpenAPI/Swagger documentation, JaCoCo coverage reporting in CI, and structured logging — delivering a polished, observable, and well-documented system.

Sprint 2 committed 18 story points across US-09 through US-14. Every Sprint 1 retrospective action item that was achievable within the sprint was addressed before new features were added.

4.2 Improvements Applied from Sprint 1 Retrospective

Enhanced Logging: Structured SLF4J logging added to all controllers, the service layer, the JWT filter, and the global exception handler. Covers login success/failure, task creation, deletion, authorisation failures, and unhandled exceptions — creating an auditable event trail.

Strengthened JWT Filter: Hardened to handle malformed tokens, expired tokens, and missing Authorization headers gracefully, each producing HTTP 401 with a descriptive message. Expiry validation confirmed via a dedicated test case.

Spring Boot Actuator: Added to pom.xml and configured to expose /actuator/health without authentication, returning {"status": "UP"} — compatible with container orchestration health probes.

Springdoc OpenAPI / Swagger UI: Auto-generates an OpenAPI 3.0 specification from controller annotations. Swagger UI accessible at /swagger-ui.html with interactive try-it-out functionality for all endpoints.

JaCoCo in CI: CI job now generates an HTML coverage report and uploads it as the named artifact jacoco-coverage-report after every run. Provides a per-commit coverage record downloadable from the Actions run summary.

Maven Dependency Caching: actions/cache@v4 added to the CI workflow, keyed on the SHA-256 hash of pom.xml. Substantially reduced pipeline duration on repeat builds.

Dashboard Auto-redirect: dashboard.html updated to detect absent tm_token in localStorage and redirect to login.html automatically, removing the need for manual console token injection.

4.3 Sprint 2 Deliverables Completed

login.html: HTML5/CSS/JS login form. On submission issues an AJAX POST to /api/auth/login, stores the returned JWT in localStorage as tm_token, and redirects to dashboard.html. On HTTP 401 displays an inline error without page reload. No external JS libraries required.

dashboard.html: On load checks for tm_token and redirects to login if absent. Fetches and renders the task list as cards showing title, description, and status. Provides a create-task form, a status-toggle button (marks task COMPLETED), and a delete button that calls DELETE /api/tasks/{id} and removes the card on success. All calls include Authorization: Bearer.

DELETE /api/tasks/{id}: Returns HTTP 204 on successful deletion. Returns HTTP 403 if the requesting user does not own the task. Returns HTTP 404 if the task does not exist. Owner-verification logic encapsulated in TaskService.

Actuator Health: GET /actuator/health returns {"status":"UP"} in normal operation. Accessible without credentials.

OpenAPI Documentation: Available as JSON at /v3/api-docs and as interactive UI at /swagger-ui.html. All six API endpoints documented with request and response schemas.

JaCoCo Coverage: Minimum line-coverage threshold configured in pom.xml. Build fails if coverage drops below threshold, preventing silent regression. HTML report generated per CI run.

Sprint 2 Documentation: Sprint2Review.md and Sprint2Retro.md created in docs/ within the repository.

4.4 Sprint 2 Scope Status

Deliverable	Status
login.html — AJAX auth, localStorage JWT persistence	Done
dashboard.html — list, create, update status, delete	Done
DELETE /api/tasks/{id} with owner checks (204/403/404)	Done
/actuator/health	Done
Springdoc OpenAPI / Swagger UI	Done
JaCoCo coverage in CI with artifact upload	Done
Structured logging across all layers	Done
JWT filter hardening and expiry test	Done
Maven dependency caching in CI	Done
Sprint 2 review and retrospective documentation	Done
E2E UI tests	Not Done — deferred SP3
JWT secret moved to environment variable	Not Done — deferred SP3
Coverage badge via Codecov	Not Done — deferred SP3
IaC and CD deployment pipeline	Not Done — deferred SP3

4.5 Sprint 2 Review Summary

All six committed Sprint 2 stories met their acceptance criteria. The review demonstrated: a user completing the full workflow (register, login via browser, create tasks, update status, delete tasks) without touching the browser console; /actuator/health returning {"status":"UP"}; Swagger UI listing all endpoints with interactive documentation; and the GitHub Actions build producing a downloadable JaCoCo coverage report.

4.6 Sprint 2 Retrospective

What Improved in Sprint 2:

- Browser UI provides a complete task management workflow without any console interaction.

- DELETE endpoint fully implemented with owner verification and test coverage for happy and failure paths.
- CI collects JaCoCo coverage on every run, creating a per-commit coverage record.
- Actuator provides a standards-compliant health endpoint ready for container health probes.
- Springdoc eliminates documentation drift — API docs stay in sync with the code automatically.
- Structured logging makes operational debugging practical across all layers.
- Maven caching reduced CI pipeline duration on repeat builds.

What Didn't Go Well / Risks Remaining:

- E2E UI tests not implemented; UI flows verified manually only. This is a coverage gap.
- JWT secret remains in application.properties. Migration to environment variables is critical before any production deployment.
- CI failure due to JaCoCo/JDK incompatibility was not resolved before sprint close.
- No CD pipeline; deployment beyond local Docker is entirely manual.

Sprint 2 Action Items:

Action Item	**Owner**	**Due**
Add E2E tests using Cypress or Playwright	dev	Sprint 3
Move JWT secret to environment variable	dev / ops	Before prod
Pin GitHub Actions runner to JDK 17	dev / ops	Immediate
Add JaCoCo coverage badge via Codecov	dev	Sprint 3
Define IaC and CD pipeline	dev / ops	Sprint 3

5. CI/CD & DevOps Documentation

5.1 CI/CD Workflow

The pipeline is defined in .github/workflows/ci.yml and triggers on push and pull_request to main.

Pipeline stages:

1. Checkout (actions/checkout@v4) — clones the repository at the triggering commit SHA.
2. Java Setup (actions/setup-java@v4) — installs Eclipse Temurin JDK 17, matching the <release>17</release> compiler target.
3. Maven Cache (actions/cache@v4) — caches ~/.m2/repository keyed on the SHA-256 hash of pom.xml.
4. Build (mvn package) — compiles source files and produces the executable JAR.
5. Test (mvn test) — executes all JUnit 5 tests with JaCoCo bytecode instrumentation.
6. Coverage Upload (actions/upload-artifact@v4) — uploads target/site/jacoco/ as jacoco-coverage-report. Persists for 30 days and is downloadable from the Actions run summary.

ci.yml:

```yaml
name: Java CI
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Cache Maven packages
        uses: actions/cache@v4
        with:
          path: ~/.m2
          key: ${{ runner.os }}-m2-${{ hashFiles('/pom.xml') }}
      - name: Build
        run: mvn package
      - name: Test
        run: mvn test
      - name: Upload coverage report
```

```
uses: actions/upload-artifact@v4
with:
  name: jacoco-coverage-report
  path: target/site/jacoco/
```

5.2 Dockerfile

Two-stage build pattern to produce a minimal, reproducible runtime image.

dockerfile

```
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /app

COPY pom.xml .

RUN mvn dependency:go-offline -B

COPY src ./src

RUN mvn package -DskipTests


FROM eclipse-temurin:17-jre-jammy

WORKDIR /app

COPY --from=build /app/target/taskmanager-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

The build stage copies pom.xml before source code to exploit Docker layer caching unchanged dependencies are not re-downloaded. The runtime stage uses a slim JRE image (no JDK toolchain), substantially reducing the final image size and attack surface.

Build and run:

```
mvn package -DskipTests

docker build -t taskmanager:latest .

docker run -p 8080:8080 taskmanager:latest
```

5.3 Coverage Reporting (JaCoCo)

JaCoCo operates in two phases. During prepare-agent it injects a Java agent that instruments bytecode at runtime and records executed lines and branches. During report it reads target/jacoco.exec and generates the HTML report in target/site/jacoco/.

The HTML report provides package-level summary, class-level detail, and line-by-line source colour coding: green (covered), red (uncovered), yellow (partially covered branch). A minimum line-coverage threshold in pom.xml fails the build if coverage drops below the configured percentage, enforcing coverage as a quality gate rather than a passive metric.

6. Testing Documentation

6.1 **Test Strategy**

| **Unit Tests** | JUnit 5 + Mockito. TaskService tested in isolation with mocked repositories. No Spring context; fast execution. |

| **Controller Integration Tests** | @SpringBootTest with MockMvc. Full application context loaded including security filter, H2 database, controllers, and services. |

| **Security Tests** | Protected endpoints verified to return 401 without token, 403 on cross-user access, and correct data with a valid token. |

| **Negative / Boundary Tests** | Duplicate registration (409), invalid login (401), task not found (404), cross-user delete (403) — all explicitly tested. |

| **E2E UI Tests** | Not implemented. Planned for Sprint 3 using Cypress or Playwright. |

6.2 **Test Files**

AuthControllerIntegrationTest

- Successful registration → HTTP 200 with username.
- Duplicate username → HTTP 409.
- Successful login → HTTP 200 with non-empty token.
- Wrong password → HTTP 401.
- Non-existent username → HTTP 401.

TaskControllerIntegrationTest

- Create task → HTTP 201 with task JSON.
- List tasks → HTTP 200 with array containing only the calling user's tasks.
- Update status → HTTP 200 with updated task.
- Delete task (owner) → HTTP 204; task absent from subsequent list.
- Delete task (non-owner) → HTTP 403.
- Protected endpoint without token → HTTP 401.

TaskServiceTest

- createTask : task saved with correct owner and PENDING status.
- listTasks : only tasks matching the user's ID returned.
- updateTaskStatus : status updated and persisted.
- deleteTask (owner) : repository delete called.
- deleteTask (non-owner) : UnauthorisedException thrown; delete not called.

7. **Monitoring & Logging**

7.1 **Actuator Configuration**

management.endpoints.web.exposure.include=health

management.endpoint.health.show-details=always

```

GET /actuator/health returns {"status":"UP"} when healthy and {"status":"DOWN"} with component detail when a dependency (e.g. the H2 datasource) reports an issue. Accessible without authentication. Additional endpoints (info, metrics, env) are available but not exposed by default, minimising attack surface.

### 7.2 **Logging Strategy**

SLF4J with Logback (Spring Boot default). Log levels applied:

- INFO — Business events: registration, login success, task created, task deleted.

- WARN — Security events: failed login attempts, authorisation failures (403).

- ERROR — Unhandled exceptions captured by the global exception handler; full stack trace logged.

- DEBUG — Detailed operational data (JWT parsing steps, query parameters); disabled by default, enabled via application.properties.

In production, the log format should be switched to JSON for compatibility with centralised log management platforms (ELK, Datadog, etc.).
```

8. **Final Reflection**

8.1 **Agile Learnings**

Sprint 0 as a distinct planning phase proved its value immediately. Investing time in backlog refinement, acceptance criteria definition, and architecture design before writing code eliminated the most common source of mid-sprint rework: ambiguous requirements and undeclared story dependencies. The backlog table served as a stable contract throughout, enabling confident sprint planning without constant re-estimation.

The Definition of Done prevented silent quality drift. Without an explicit DoD, different team members apply different standards — one considers a feature done when the happy path works; another requires tests and CI to pass. The shared DoD created an objective gate that prevented half-finished work from accumulating.

Retrospectives as a formal ceremony, rather than an informal debrief, produced action items with owners and due dates. The Sprint 1 retrospective directly generated the Sprint 2 improvements in logging, JWT hardening, and UI authentication workflow, demonstrating that the retrospective-to-action cycle works when outcomes are tracked rather than discussed and forgotten.

8.2 **DevOps Learnings**

CI is not a one-time setup task but a living component that requires maintenance as the project evolves. The JaCoCo/JDK incompatibility illustrates that tooling combinations can fail even when each component is individually functional. A red CI build must be treated as a blocking condition — not a background concern — otherwise the pipeline loses its value as a reliable signal.

Containerisation via Docker clarified the distinction between build and runtime environments. The two-stage Dockerfile pattern produces reproducible, minimal images: smaller attack surface, faster pull times, and consistent behaviour regardless of the developer machine. These benefits are not theoretical; they were directly

observable when the Docker image ran correctly on a clean environment without any local Java installation.

Maven dependency caching demonstrated that small pipeline optimisations compound. A single cache step substantially reduced build time on unchanged code, improving the feedback loop without changing any application logic. DevOps improvement is not always about adding capability; often it is about removing friction.

8.3 **CI/CD Lessons**

The CI pipeline was most valuable when it failed. The JaCoCo/JDK failure surfaced a real environmental incompatibility that would have been much harder to diagnose without the automated run record. Every failure is information; the pipeline's job is not to succeed but to provide an accurate signal about the current state of the codebase.

The absence of a CD pipeline is the most significant remaining gap. The current system requires a developer to manually build the Docker image and push it to any environment. Implementing a CD stage that automatically tags and pushes an image to a container registry (GitHub Container Registry, DockerHub) and deploys it to a staging environment on every successful main-branch build would complete the DevOps lifecycle.

8.4 **Biggest Improvements Across Sprints**

The most substantial improvement from Sprint 1 to Sprint 2 was the transition from an API-only system to a browser-usable application. Sprint 1 delivered correctness; Sprint 2 delivered usability. The login and dashboard pages transformed the prototype from a tool requiring curl knowledge into something any stakeholder can evaluate directly.

JaCoCo coverage reporting changed the relationship between the team and test quality. Before coverage metrics, tests were written because the DoD required them. After, coverage became a quantified, per-commit metric — an objective baseline for identifying gaps and tracking improvement.

Structured logging, while invisible to end users, represents a significant leap in operational maturity. A system that emits structured, searchable events is qualitatively different from one that is silent, regardless of functional behaviour. Sprint 2 laid the observability foundation required for any credible production deployment.

8.5 **What Would Be Done Differently**

E2E UI tests from day one. Cypress or Playwright tests for the browser flows should have been planned in Sprint 0 and implemented in Sprint 2. Their absence leaves a verification gap between what the system does and what is automatically confirmed.

JWT secret externalisation in Sprint 1. A hardcoded secret should have been moved to an environment variable in the same sprint that introduced JWT authentication. Security fundamentals are not a deferred concern.

Pagination for the task list. GET /api/tasks returns all tasks without pagination. This is a performance and UX issue at any meaningful data scale. Cursor-based pagination is a Sprint 3 priority.

CD pipeline alongside CI. A CD job that deploys the Docker image to a free-tier cloud environment (Fly.io, Railway) would have provided a live demo URL and completed the DevOps lifecycle, turning the project from a local prototype into a deployed service.

Coverage threshold from Sprint 1. The JaCoCo minimum threshold should have been enforced from the first sprint, making coverage a non-negotiable quality gate from the start rather than a Sprint 2 addition.

8.6 **Conclusion**

The Taskmanager project delivered a functioning, observable, and testable task management system across three structured sprints. The system progressed from an empty repository to a JWT-authenticated REST API with a browser UI, automated CI

pipeline, containerisation, health monitoring, and self-documenting API specification — a meaningful scope of work within a semester timeframe.

The project validated several principles that are difficult to internalise without direct experience: that a working Definition of Done prevents quality drift; that automated tests pay back their cost immediately when they catch regressions; that a CI pipeline is only valuable if kept green; and that retrospective action items only have value when they are actually actioned. Each principle was tested by a real failure mode encountered during the project and addressed through the structured Agile process.

The remaining work — E2E tests, secret management, CD pipeline — reflects honest sprint planning rather than incomplete delivery. The team committed to what could be delivered with confidence and deferred what could not, producing a system that is correct and demonstrable rather than superficially complete but fragile.

Appendix

A. **Sprint Plans Summary**

Sprint 0 —> Planning

Outputs: full product backlog, Definition of Done, architecture design, Sprint 1 plan.

Sprint 1 — > Backend Foundation

- Committed points: 17
- Stories: US-01 through US-08
- Goal: JWT-authenticated REST API with tests, CI, and Docker packaging
- Outcome: All stories delivered; UI and observability deferred to Sprint 2

Sprint 2 — > Execution & Improvement

- Committed points: 18
- Stories: US-09 through US-14
- Goal: Browser UI, delete API, Actuator, Swagger, JaCoCo in CI, structured logging
- Outcome: All stories delivered; E2E tests and CD pipeline deferred to Sprint 3

B. **API Endpoint Summary**

| Method | Path | Auth | Response | Sprint |

| POST | /api/auth/register | None | 200 / 409 | SP1 |

| POST | /api/auth/login | None — returns JWT | 200 {token} / 401 | SP1 |

| GET | /api/tasks | Bearer JWT | 200 array | SP1 |

| POST | /api/tasks | Bearer JWT | 201 task | SP1 |

| PUT | /api/tasks/{id}?status=... | Bearer JWT | 200 task / 403 | SP1 |

| DELETE | /api/tasks/{id} | Bearer JWT (owner) | 204 / 403 / 404 | SP2 |

| GET | /actuator/health | None | 200 {status} | SP2 |

| GET | /swagger-ui.html | None | 200 HTML | SP2 |

| GET | /v3/api-docs | None | 200 JSON spec | SP2 |

C. **Key Configuration Notes**

Running locally:

mvn -Djacoco.skip=true test run tests (JDK 17 or 21)

mvn spring-boot:run start application on port 8080

Known issue: JaCoCo 0.8.10 fails with JDK versions newer than 21. Pin the GitHub Actions runner to JDK 17, or add -Djacoco.skip=true to the CI test command, to resolve.

Production readiness checklist:

- Move JWT secret from application.properties to an environment variable or secrets manager.
- Replace H2 with a persistent database (PostgreSQL or MySQL).
- Implement a CD pipeline to automate deployment.
- Add E2E UI tests to close the browser-flow verification gap.
- Switch log format to JSON for centralised log aggregation.